

Script oral — Partie 1

Réseaux de neurones pour la résolution d'EDP — Méthode PINN

Paul Rivaud • Projet 1A, CERMICS • 6 juin 2026 • durée visée : ≈ 4 min 30

Ton & posture. Débit posé, regard vers le jury (pas vers les slides), une respiration entre chaque diapo. Les mots **en gras** sont à appuyer. Les formules sont écrites *telles qu'on les prononce* : lisez-les à voix haute sans les déchiffrer.

Diapo 1 — Titre

(≈ 20 s)

Bonjour à tous. Nous vous présentons notre projet de première année, encadré par Sofiane Ezzehi au laboratoire CERMICS : **la résolution d'équations aux dérivées partielles par réseaux de neurones**. Je m'occupe de cette première partie : je pose le problème, je situe l'état de l'art, puis je présente la méthode au cœur de notre travail — les **Physics-Informed Neural Networks**, les PINNs.

Diapo 2 — Plan de la présentation

(≈ 15 s)

Voici le déroulé. Dans cette première partie, j'introduis le contexte et la méthode PINN. Mes camarades enchaîneront ensuite sur la **méthode Deep Ritz** et notre approche, puis sur les **difficultés et le code**, et enfin sur les **résultats numériques** et la conclusion.

Diapo 3 — Contexte & motivation

(≈ 30 s)

Les équations aux dérivées partielles sont partout : elles décrivent la diffusion de la chaleur, la mécanique des structures, ou encore certains modèles financiers. Le problème, c'est que la quasi-totalité d'entre elles **n'ont pas de solution analytique** : il faut donc les résoudre numériquement. La question qui guide tout notre projet est alors la suivante : **peut-on résoudre une EDP en exploitant la puissance des réseaux de neurones, sans passer par un maillage du domaine ?**

Diapo 4 — Les méthodes classiques et leurs limites

(≈ 35 s)

Rappelons d'abord comment on procède classiquement. Les méthodes traditionnelles — différences finies, éléments finis — reposent toutes sur une **discrétisation** : on découpe le domaine en un **maillage**, et on résout l'équation sur ce maillage. C'est très efficace en petite dimension, mais le coût **explose en grande dimension** — c'est le fléau de la dimension — et le maillage devient délicat pour des géométries complexes. À droite, l'idée opposée : au lieu d'un maillage régulier, le réseau travaille à partir de points dispersés, dits **points de collocation**. Comme fil rouge de tout l'exposé, nous prendrons l'équation de Poisson en une dimension : *moins u seconde égale f* , avec des conditions de Dirichlet fixées aux deux bords.

Diapo 5 — Pourquoi des réseaux de neurones ?

(≈ 35 s)

Pourquoi penser que cela peut marcher ? Pour deux raisons. La première est théorique : le **théorème d'approximation universelle**, démontré par Cybenko en 1989, garantit qu'un réseau à **une seule couche cachée**, avec une activation continue, peut approcher **uniformément** n'importe quelle fonction continue. Autrement dit, il est légitime de chercher notre solution sous la forme d'un réseau. La seconde raison est l'idée maîtresse des PINNs : on **incorpore directement l'équation et les conditions aux bords dans l'apprentissage**. C'est un cadre **non supervisé**

— aucune donnée étiquetée, aucune solution connue à l’avance : le réseau apprend en minimisant le **résidu** de l’équation elle-même.

Diapo 6 — Principe des PINN

(≈ 35 s)

Concrètement, l’idée des PINNs — introduite par Raissi, Perdikaris et Karniadakis en 2019 — est la suivante : **le réseau u_θ est notre solution approchée**. Comme c’est une fonction explicite, on peut calculer ses dérivées **exactement**, par **différentiation automatique** — la même mécanique que celle qui sert à entraîner les réseaux. On évalue alors le **résidu** de l’équation, *r égale u -thêta seconde plus f* , et on transforme ainsi l’EDP en un **problème d’optimisation**. Le schéma résume la boucle : on part des points, on les passe dans le réseau, on dérive, on calcule le résidu puis la perte, et la **rétropropagation** met à jour les paramètres — ici avec l’optimiseur Adam.

Diapo 7 — Architecture du réseau

(≈ 30 s)

Voici l’architecture retenue. Une entrée scalaire x , une **couche cachée** de n neurones avec une activation **tangente hyperbolique**, et une sortie : la valeur $u_\theta(x)$. L’ensemble des fonctions ainsi représentables forme l’espace V_n , paramétré par $3n$ réels. Le choix de la tangente hyperbolique n’est pas anodin : elle est **indéfiniment dérivable**, donc notre solution est bien de classe C^2 , et surtout sa **dérivée seconde est calculable proprement** par différentiation automatique — ce qui est indispensable pour une équation du second ordre.

Diapo 8 — La fonction de perte physique

(≈ 35 s)

On arrive au cœur de la méthode : la **fonction de perte**. On définit une **fonctionnelle d’énergie** qui mesure à quel point le réseau viole l’équation à l’intérieur du domaine, augmentée de deux termes qui pénalisent le non-respect des conditions aux bords. En pratique, l’intégrale est approchée par une **moyenne sur N points de collocation** : la perte se lit en deux morceaux, le terme physique L -PDE et le terme de bord L -BC. Point clé : la **solution exacte est exactement celle qui annule cette énergie**. Entraîner le réseau revient donc à minimiser la perte par **descente de gradient** — Adam — le gradient étant fourni par rétropropagation.

Diapo 9 — Conclusion de partie & transition

(≈ 20 s)

Pour résumer cette première partie : les PINNs **reformulent une EDP en un problème d’optimisation**, où le réseau apprend à **annuler le résidu** de l’équation. Les atouts sont nets : **pas de maillage**, apprentissage **non supervisé**, et une approche qui passe bien à l’échelle en grande dimension et pour les problèmes inverses. Je laisse maintenant la parole à [prénom du camarade] pour la méthode Deep Ritz et notre implémentation. Merci de votre attention.